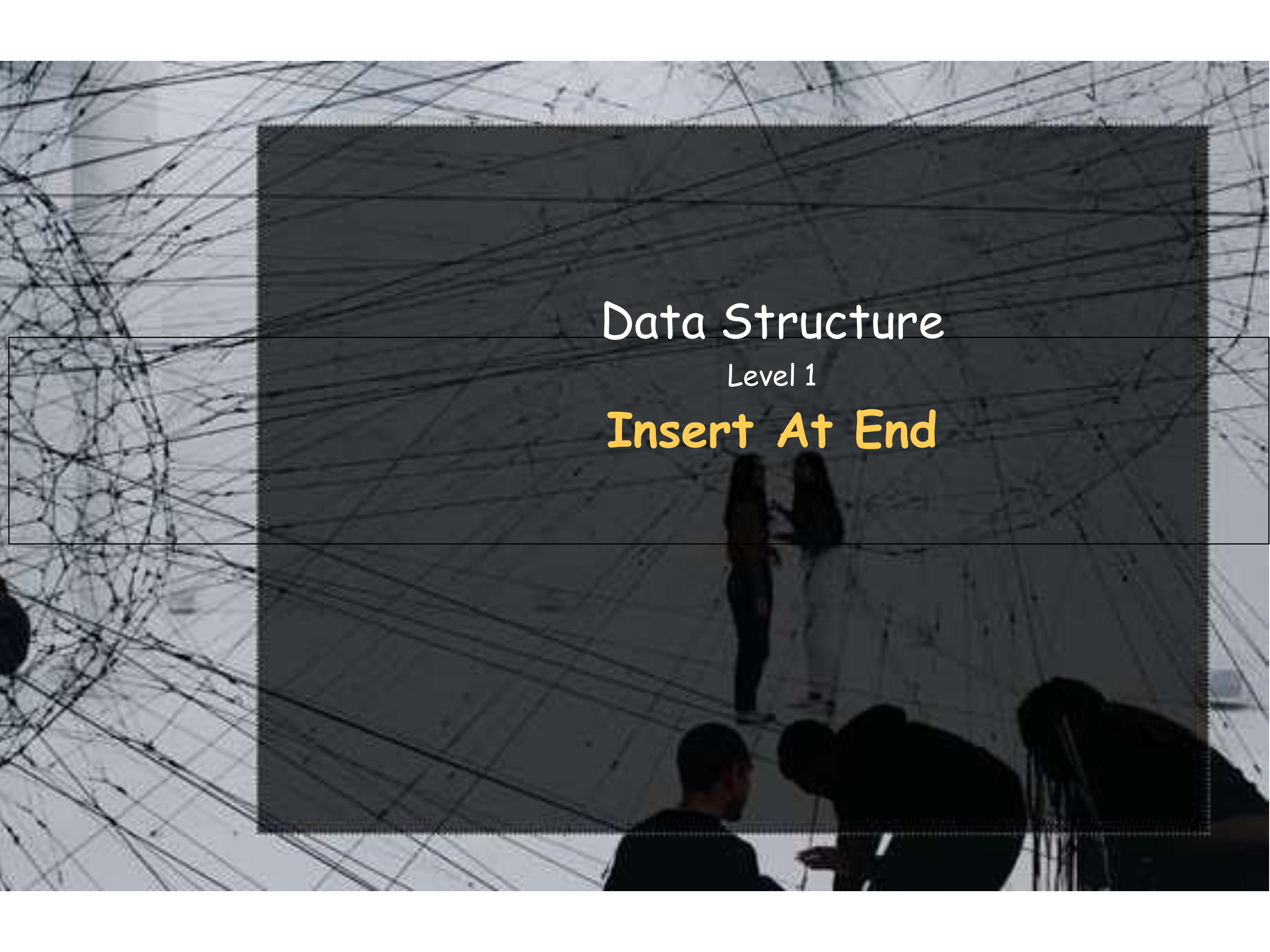


Data Structure

Level 1

Singly Linked List Data Structures Operations

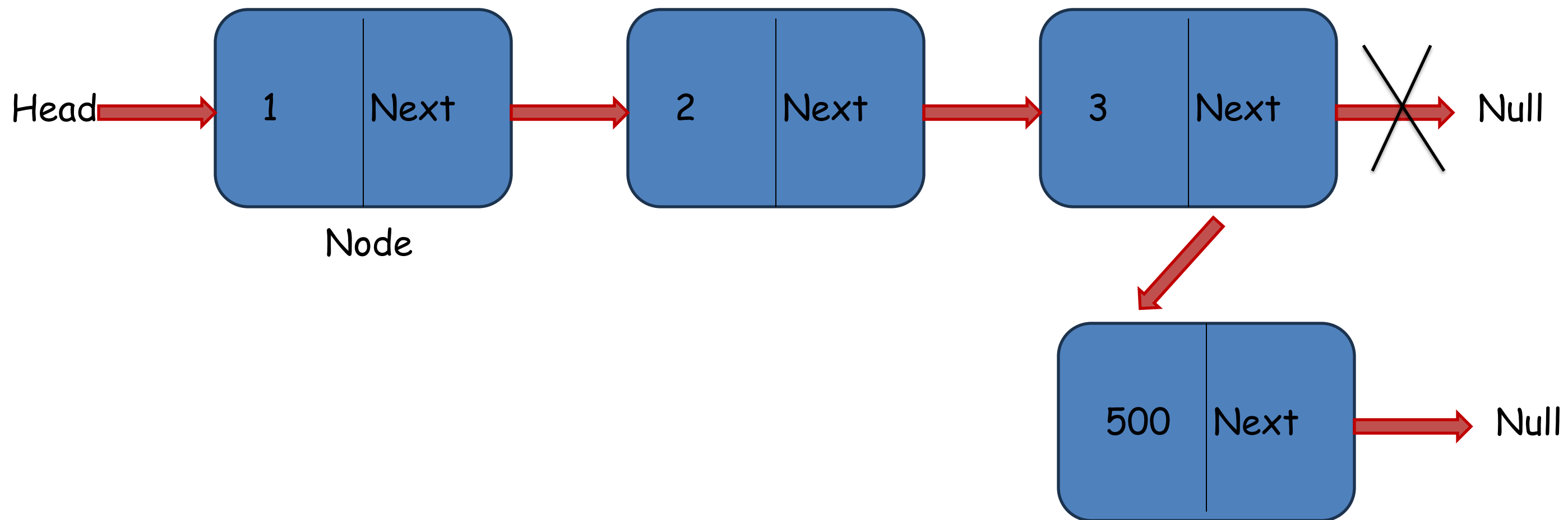


Data Structure

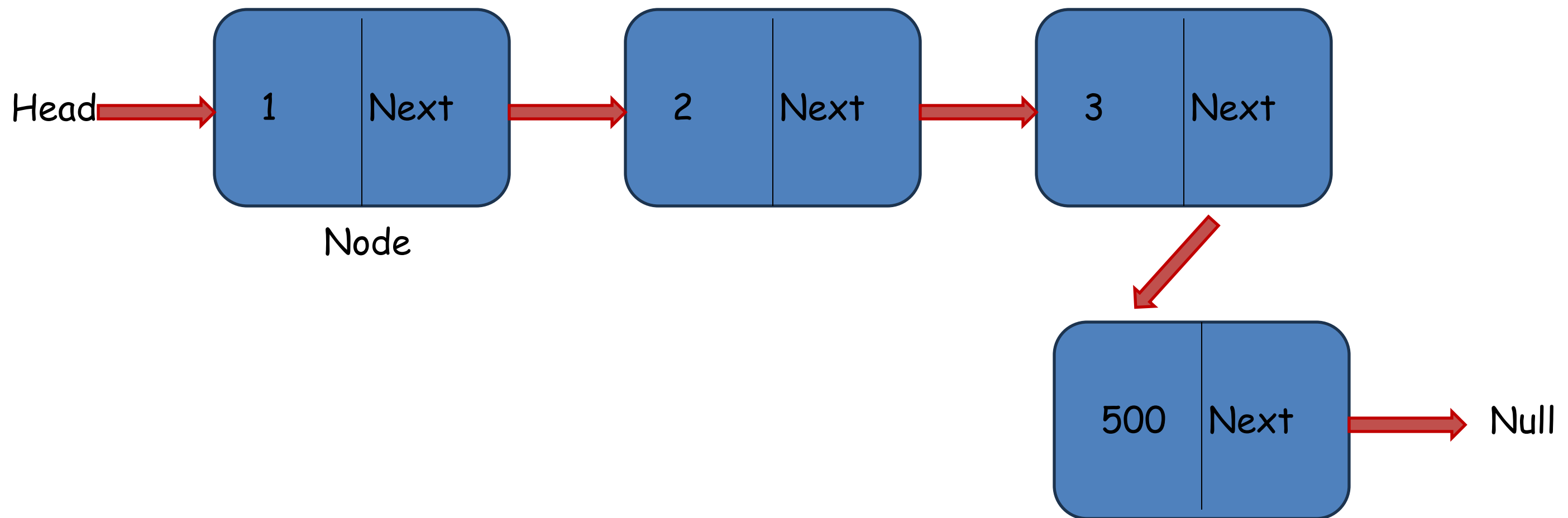
Level 1

Insert At End

Linked List Data Structure



Linked List Data Structure





Definition

InsertAtEnd() Function:

Adds a new node at the end of the linked list.

Behavior:

- Creates a new node
- Sets its next to NULL
- Traverses to the last node
- Links the last node to the new node



Explanation

- ◆ If the list is empty (`head == NULL`), the new node becomes the head.
- ◆ Otherwise, we iterate through the list until `LastNode->next == NULL`.
- ◆ The last node's next pointer is updated to the new node.
- ◆ This preserves the linked structure.

Procedures (Insert at End)

- 1 Create a new node
- 2 Assign value to the node
- 3 Set new_node->next = NULL
- 4 Check if list is empty
- 5 If empty → set head = new_node
- 6 If not empty → traverse to last node
- 7 Move pointer until next == NULL
- 8 Set LastNode->next = new_node
- 9 Print the updated list

Example

Operations in main():

InsertAtEnd(1)

InsertAtEnd(2)

InsertAtEnd(3)

InsertAtBeginning(0)

➡ Final List:

0 → 1 → 2 → 3 → NULL



Note

Insertion at end requires traversal, unlike insertion at beginning.



Comparison

■ Insert at Beginning

- ✓ $O(1)$ time
- ✓ No search required

■ Insert at End

- ✓ Time Complexity: $O(n)$
- ✓ Requires traversal to last node

Warning

If you forget to set `new_node->next = NULL`, the list may contain garbage values and cause errors.

✨ Summary

- ✓ InsertAtEnd adds node to the end of the list
- ✓ Requires traversal if list is not empty
- ✓ Time complexity is $O(n)$
- ✓ Can be optimized using a tail pointer

Thank you 😊